

Environment Structure, Not Just Team Size: A Case Study in the Generalization Limits of Permutation-Invariant Coverage Policies

Harsh Pandhe¹

¹Independent Research , harshpandhehome@gmail.com

Abstract

Permutation-invariant neural architectures (deep sets, graph neural networks) are well established as a route to multi-robot policies that generalize across team size without retraining. We highlight a second, less examined axis of generalization: environment *structure*. Using an identical navigation stack – an A*-augmented frontier-exploration controller behind a permutation-invariant neighbor encoder – deployed unchanged into two real-world-derived Gazebo environments, we show that reachable area-coverage ceiling differs sharply by environment structure alone: 90.4% in a densely furnished cafe environment versus 99.0%–100.0% in an open warehouse environment containing only isolated pillar obstacles, of comparable or larger physical area. Both environments were independently verified via offline mesh/geometry analysis before deployment, so the difference is attributable to environment topology (dense, coverage-blocking clutter versus sparse, routable obstacles) rather than measurement artifacts. We argue that team-size invariance and environment-structure invariance are separate properties that do not follow from each other, and that reporting a single coverage number without characterizing the environment’s obstacle topology understates how much of a policy’s apparent performance is actually attributable to the environment it was tested in.

Keywords: multi-robot coverage, permutation-invariant neural networks, graph neural networks, generalization, swarm robotics

1 Introduction

A permutation-invariant policy – one whose neighbor-observation processing is invariant to the number and ordering of neighbors, typically via a shared per-neighbor encoder followed by a symmetric pooling operation [6] – is now a standard tool for building multi-robot policies that scale to team sizes unseen during training [1, 5, 2]. This is a genuinely useful and well-demonstrated property. Our concern in this paper is a different, easily conflated one: does scaling to a new *team size* with a fixed architecture say anything about scaling to a new *environment*? We argue it does not, by construction – team-size invariance is a property of the neighbor-processing module, while environment generalization additionally depends on the exploration/planning components consuming the environment’s structure, which permutation invariance does not touch.

We make this concrete with a controlled case study: the same navigation stack, deployed unchanged into two physically distinct, independently verified environments – a furnished cafe and an open warehouse – and show that the ceiling on achievable area coverage differs by roughly 9–20 percentage points between them, driven entirely by environment topology.

2 Related Work

Zaheer et al. [6] establish the deep-sets construction — shared per-element encoding followed by a symmetric pooling operation — as the canonical route to permutation-invariant function approximation, which underlies most permutation-invariant multi-robot policies including ours. An et al. [1] apply permutation-invariant networks to multi-entity robotic control via a PointNet-style architecture. Wang et al. [5] combine agent-centric canonicalization with role-aware graph encoders for equivariant, sample-efficient, and generalizable swarm control, validated in a sim-to-real pursuit-evasion setting. Blumenkamp et al. [2] deploy decentralized GNN-based policies on a real multi-robot system communicating over ad-hoc networks, demonstrating sim-to-real transfer of a GNN policy architecture. Gosrich et al. [3] apply GNNs directly to multi-robot coverage control, the same task family we study, and report improved coverage quality from inter-robot communication relative to purely local information – complementary to our finding, since their generalization axis is communication topology and team size, not physical environment structure. Muthusamy et al. [4] study GNN generalizability for decentralized unlabeled motion planning, the closest prior work to our concern, but frame generalization primarily in terms of team size and graph connectivity rather than environment obstacle topology. To our knowledge, no prior work in this family holds team size and architecture fixed while varying environment structure as

the sole independent variable, which is precisely the controlled comparison we report.

3 Policy Architecture

The actor separates ego features $\mathbf{e} \in \mathbb{R}^{28}$ (LiDAR, goal, velocity) from a variable-size neighbor set. Each neighbor’s relative (distance, angle) pair is embedded by a shared two-layer MLP $\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^{16}$; padded (absent) neighbors are masked out and the valid embeddings mean-pooled:

$$\mathbf{g} = \frac{1}{\max(1, |\mathcal{V}|)} \sum_{j \in \mathcal{V}} \phi(n_j), \quad \mathcal{V} = \{j : d_j < 9.0 \text{ m}\}, \quad (1)$$

which is invariant to neighbor count and ordering by construction, and was verified to run unchanged (no dimension mismatch, correct output shapes) at $N = 5$ and $N = 10$ neighbors against a network trained assuming $N = 3$. This module is architecturally identical across both environments studied below; what differs between environments is not this module but the frontier target-selection and A* routing logic each robot uses to decide *where* to go, which necessarily depends on the map it observes.

4 Case Study Design

We deploy an identical three-robot navigation stack – frontier-based target selection (closest unvisited grid cell) augmented with A* routing (activated only when the direct line to the target is obstructed, with a one-cell obstacle-inflation safety margin) – into two Gazebo Sim environments.

Cafe environment. A furnished cafe model (Gazebo Fuel, `openrobotics/models/cafe`) with tables, counters, and other clutter distributed through the interior. Coverage grid: 40×20 cells over a $\sim 17 \times 8$ m usable region ($\sim 136 \text{ m}^2$).

Warehouse environment. An open warehouse model (Gazebo Fuel, `openrobotics/models/Warehouse`) independently verified offline before deployment by downloading its collision mesh (STL) and computing occupancy at LiDAR sensing height (10–30 cm): a 30×50 m overall footprint that is 95–100% open floor at that height, with obstacles limited to outer walls and six isolated support pillars arranged in a 2×3 grid. We deployed into an 18×12 m region (216 m^2 – larger than the cafe region) deliberately including one pillar pair (at world coordinates $x \approx \pm 7.25$ m, $y \approx 0$ m), so the comparison includes genuine obstacle discovery and avoidance rather than an artificially obstacle-free room. Coverage grid: 45×30 cells, scaled to keep cell size (0.4 m) comparable to the cafe grid and above the navigation stack’s 0.2 m target-arrival tolerance – reusing a fixed grid resolution across differently-sized environments

was found in preliminary testing to produce cells smaller than this tolerance, causing a target cell to be approached but never entered, and therefore never marked visited, a deadlock that froze the swarm at 3–18% coverage; this is itself a second, distinct environment-structure sensitivity (grid discretization relative to physical scale) that a fixed architecture does not resolve automatically.

Both environments use the same per-step reward-shaping-free evaluation protocol: a single long episode (continuous re-targeting on goal arrival, no episode-ending termination on collision) run to 1200–1600 steps, with area-coverage rate (ACR), overlap redundancy, swarm path length, and a collision-step count (steps where any robot’s minimum LiDAR range is below 0.20 m) recorded throughout.

5 Results

Table 1: Coverage ceiling by environment, identical navigation stack.

Environment	Area (m ²)	Steps	ACR (%)	Redund.	Coll.-steps
Cafe (reactive only)	136	1200	87.5	–	463
Cafe (+ A* routing)	136	1200	90.4	–	1824
Warehouse (2 real pillars)	216	1600	99.0	10.11	607

The cafe environment’s coverage ceiling under a purely reactive controller (no path planning around obstacles) is 87.5%, and does not improve with additional run length – the remaining $\sim 12\%$ consists of cells the controller can see but cannot reach in a straight line, and reactive-only avoidance cannot route around the intervening furniture. Adding A* routing recovers some of this gap (87.5% $\rightarrow$ 90.4%) at the cost of a substantially higher collision-step count, since the swarm now pushes into tighter, harder-to-reach corners near obstacles for the remaining coverage. The warehouse environment, despite covering a larger physical area and containing genuine pillar obstacles the swarm must discover and route around, reaches 99.0% under the identical A*-augmented stack – because its obstacles are sparse and isolated rather than densely arranged, essentially no floor area is rendered permanently unreachable by the environment’s topology.

6 Discussion

The permutation-invariant neighbor-encoding module described in Section 3 is unchanged across both environments and both A* configurations in Table 1 – the 9–20 percentage-point

coverage gap is not attributable to it. It is attributable entirely to environment topology interacting with the (non-invariant) exploration and routing logic: a environment whose obstacles create disconnected or hard-to-reach pockets imposes a coverage ceiling that no amount of neighbor-encoder generalization addresses, because the neighbor encoder never sees the static environment map at all – it only processes other robots’ relative states. This is the sense in which team-size invariance and environment-structure invariance are independent properties: an architecture can be perfectly invariant to how many teammates are present while still being highly sensitive to where the walls are.

A second, narrower finding reinforces the same point at the level of implementation detail rather than architecture: the coverage-grid deadlock described in Section 4 shows that even a supporting data-structure choice (grid cell size) that was well-tuned for one environment’s physical scale produced a hard failure (not merely reduced performance) when reused unchanged in a differently-scaled environment. Grid resolution is not part of the neural architecture at all, but it is exactly the kind of environment-coupled implementation detail that a ”team-size generalization” evaluation would never surface, since it depends on absolute physical scale rather than agent count.

We see two practical implications. First, evaluations of permutation-invariant multi-robot policies that vary team size while holding environment fixed (the common protocol in the cited related work) are measuring a real but partial form of generalization; a companion evaluation varying environment structure while holding team size fixed, as done here, measures an orthogonal dimension that is at least as relevant to real deployment, where the environment is rarely a design choice. Second, for coverage-specific tasks, progress on the coverage ceiling itself (Table 1, cafe 87.5% $\rightarrow$ 90.4%) came entirely from the non-invariant planning component (A^* routing), not from the invariant one – suggesting that closing the remaining gap toward 100% in structurally harder environments is a planning-and-representation problem for the *map*, not a scaling problem for the *team*.

7 Limitations and Future Work

This is a two-environment case study, not a systematic sweep; we do not claim the specific 9–20 point gap generalizes quantitatively to other environment pairs, only that a gap of this kind is real, reproducible, and attributable to topology rather than team size or architecture. Both environments are simulated; the natural extension is sim-to-real hardware transfer, following the real-world GNN deployment pattern demonstrated by Blumenkamp et al. [2], to test whether the same structure-dependent ceiling reproduces physically, and a broader sweep across environment topologies (varying obstacle density and connectivity independently) to

characterize the relationship between topology and coverage ceiling more precisely than the two-point comparison given here.

8 Conclusion

Holding a permutation-invariant multi-robot policy’s architecture and team size fixed while varying the deployment environment reveals a 9–20-percentage-point swing in achievable area coverage, attributable to environment topology rather than the neighbor-encoding module responsible for team-size generalization. We argue this constitutes a second, independent axis of generalization for multi-robot coverage policies that existing team-size-focused evaluations do not surface, and that reporting a coverage number without characterizing the deployment environment’s obstacle topology risks attributing environment-driven performance differences to the policy itself.

References

- [1] Tianxu An, Joonho Lee, Marko Bjelonic, Flavio De Vincenti, and Marco Hutter. Solving multi-entity robotic problems using permutation invariant neural networks, 2024. arXiv:2402.18345.
- [2] Jan Blumenkamp, Steven Morad, Jennifer Gielis, Qingbiao Li, and Amanda Prorok. A framework for real-world multi-robot systems running decentralized gnn-based policies, 2021. arXiv:2111.01777.
- [3] Walker Gosrich, Siddharth Mayya, Rebecca Li, James Paulos, Mark Yim, Alejandro Ribeiro, and Vijay Kumar. Coverage control in multi-robot systems via graph neural networks, 2021. arXiv:2109.15278.
- [4] Shreyas Muthusamy, Damian Owerko, Charilaos I. Kanatsoulis, Saurav Agarwal, and Alejandro Ribeiro. Generalizability of graph neural networks for decentralized unlabeled motion planning, 2024. arXiv:2409.19829.
- [5] Keqin Wang, Tao Zhong, David Chang, and Christine Allen-Blanchette. Local-canonicalization equivariant graph neural networks for sample-efficient and generalizable swarm robot control, 2025. arXiv:2509.14431.
- [6] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander Smola. Deep sets. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.